《算法分析与设计》
课后作业

作业编号 作业 3

学　　号 _____

姓　　名 _____

专　　业 _____

学　　院 _____

第 3 章 穷举法

一、简答题（共 20 分）

1、什么是回溯法的解空间？解空间通常用什么形式表达？

2、什么是解空间树？解空间树分为哪两种类型？

3、什么是部分解、可能解、可行解和最优解？

4、什么是目标函数、约束函数和限界函数？

5、写出 0/1 背包问题的解向量结构，画出解空间树（假设 n=3）。

6、写出旅行商问题的解向量结构，画出解空间树（假设 n=4）。

7、写出图着色问题的解向量结构，画出解空间树（假设顶点数=3，颜色数=4）。

（注：解向量结构指解向量的长度和每个分量的取值范围；解空间树可以手绘并拍照附在文档中）

（略）

二、编程题 1：投资问题（共 40 分）

某公司进行组合投资，把钱同时投入到多个项目，并希望获取最大的收益。在已知该公司希望投资的最大金额数和投资的项目数的情况下，请编写程序计算其所能获得的最大收益。

输入：

输入的第一行包含两个整数 n 和 m，分别表示该公司的投资资金数和希望投资的项目数。其后的 n 行，每行有 m 个整数，分别表示将金额 1、2、…、n 单独投资到 m 个项目时的收益（投资为 0 时 m 个项目的收益均为 0）。每次组合投资必须将 n 用完，组合投资中某个项目的投资额可以为 0。

输出：

输出一行，表示该公司进行组合投资所能获得的最大收益。

样例输入：

3 2

3 2

3 5

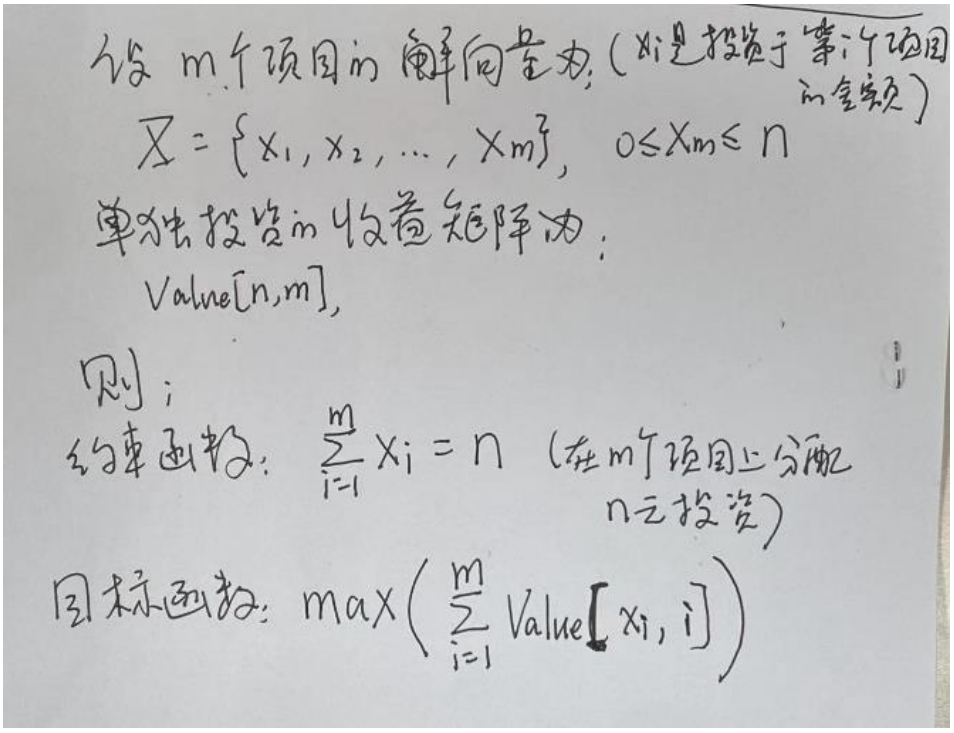
4 7

样例输出：

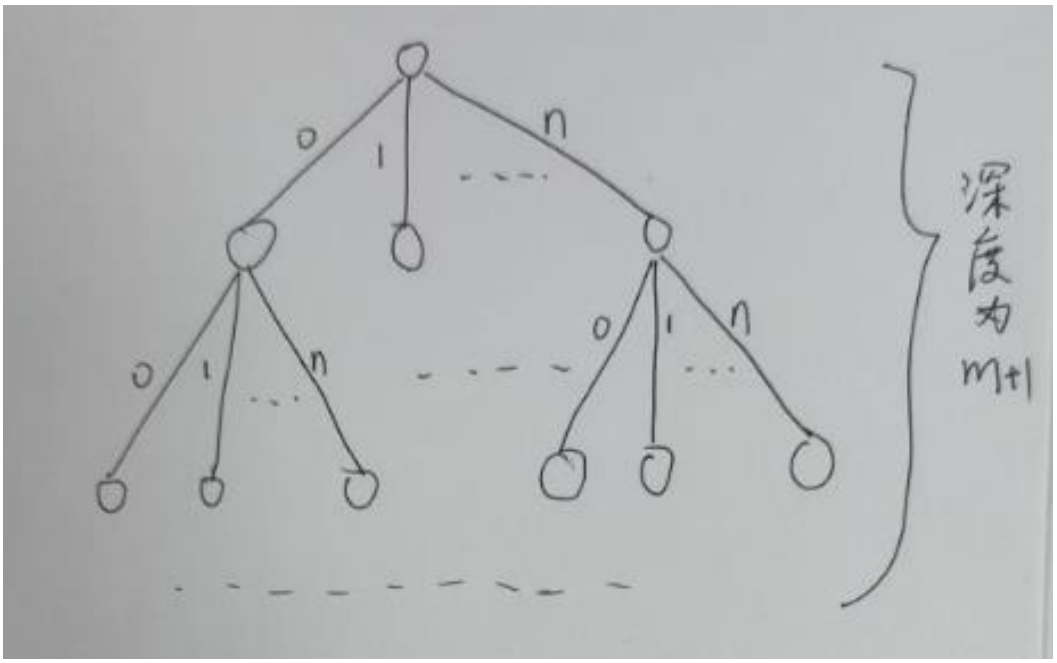
8

要求：

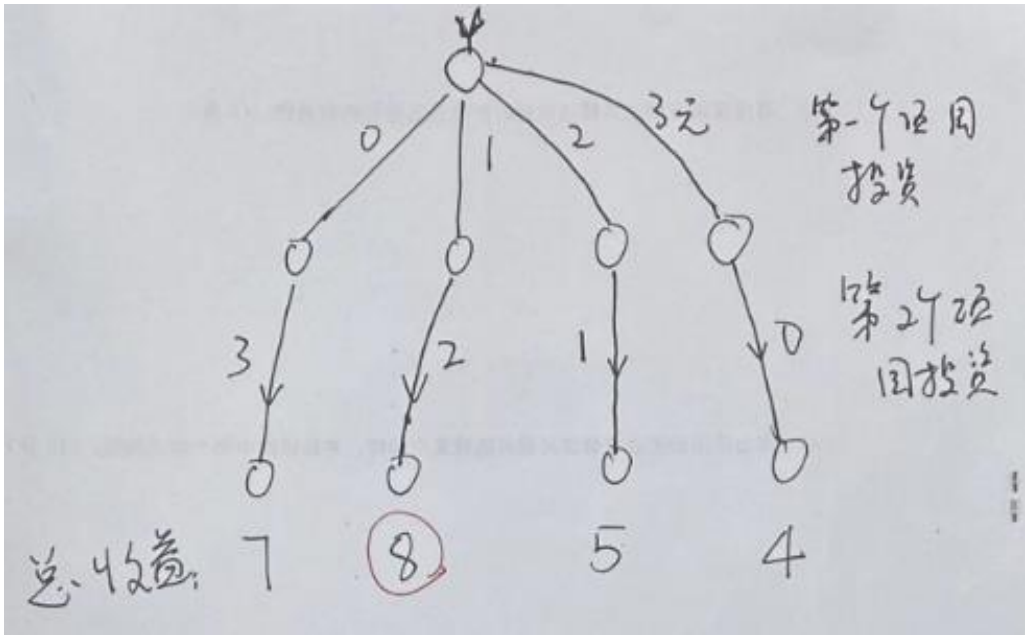
（1）写出采用回溯法求解该问题时的约束条件和目标函数。（5 分）



（2）画出该问题通用的解空间树（即与测试用例无关的解空间树，树的节点中无需标注任何信息；只画到第 2 层节点，其中根节点为第 0 层）（5 分）。



（3）采用回溯法分析上面的测试用例，画出搜索空间树，在树的节点处标明解向量的值和当前总收益。（10 分）



（4）采用回溯法编写程序实现上述题目要求，附手抄的源代码，并给出以下测试用例的输出。（20 分）

参考代码：

```
void inputAndBuild();
void dfs(int level, int curr_invest, int curr_profit);
int n, m; //n:投资总额; m: 项目数;
int max_profit; //max_profit: 组合投资最大收益
int profit[50][50];
int main()
{
    cin >> n >> m;
    max_profit = 0;
    inputAndBuild();
    dfs(0, 0, 0);
    cout << max_profit << endl;
    return 0;
}
void inputAndBuild()
{
    int c;
    int i, j;
    for (j = 0; j < m; j++) {
        profit[0][j]=0;   //每个项目投资 0 元时收益为 0
    }
```

```
    for (i = 1; i <= n; i++) {
        for (j = 0; j < m; j++) {
            cin >> c;
            profit[i][j] = c;
        }
    }
}

//level: 递归深度(即项目号，范围：0到m-1)；curr_invest：当前总投资；curr_profit：当前总收益
void dfs(int level, int curr_invest, int curr_profit)
{
    int i;
    if (level==m) {
        if ((curr_invest=n) && (curr_profit>max_profit)){
            max_profit=curr_profit;
        }
        return;
    }
    for (i=0 ; i<=n ; i++){   //横向遍历每个项目投资 0 到 n 元的情况
        if ((curr_invest+i)<=n)  //约束函数
        {
            dfs(level+1, curr_invest+i, curr_profit+profit[i][level]);
        }
    }
}
```

三、编程题 2：迷宫游戏（共 20 分）

已知有一个 n*n 迷宫，迷宫最外侧是墙壁，内部有障碍物。请编写程序完成给定功能。

输入：

（1）n*n 迷宫数组（最外层为墙壁），其中元素的含义为：2—墙壁；0—通路。

（2）迷宫入口和出口坐标。

输出：

（1）从入口到出口的路径条数；

（2）从入口到出口的最短路径长度。

要求：

（1）写出采用回溯法求解该问题时的约束条件和目标函数。（5 分）

约束函数：

（不能跨越边界） 并且（不能穿越障碍） 并且 （不能走回头路）

注意：也可以用符号表示，如

(i>=1) and (i<=n) and (j>=1) and (j<=n) and (map[i,j]==0) and !visited[i,j]

目标函数：

（1）求解路径条数：

(i == endI) and (j == endJ) and count(Path)

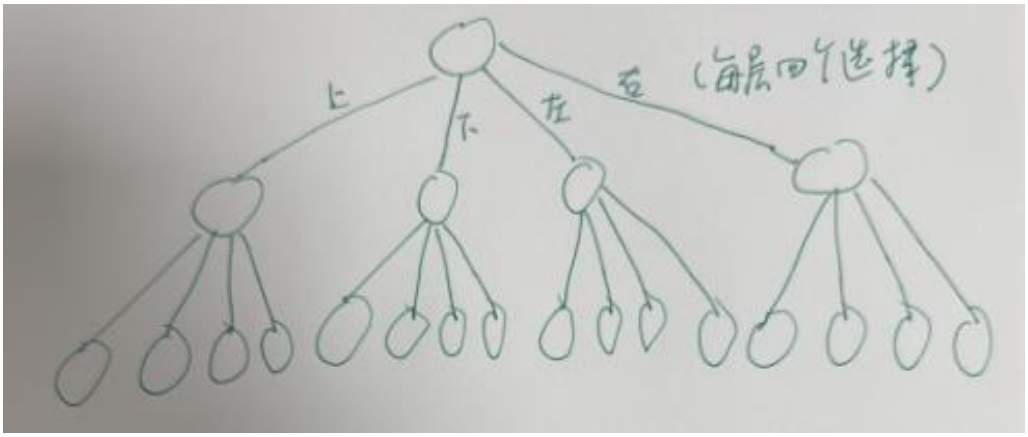
（2）求解最短路径长度

(i == endI) and (j == endJ) and (Min(PathLength)

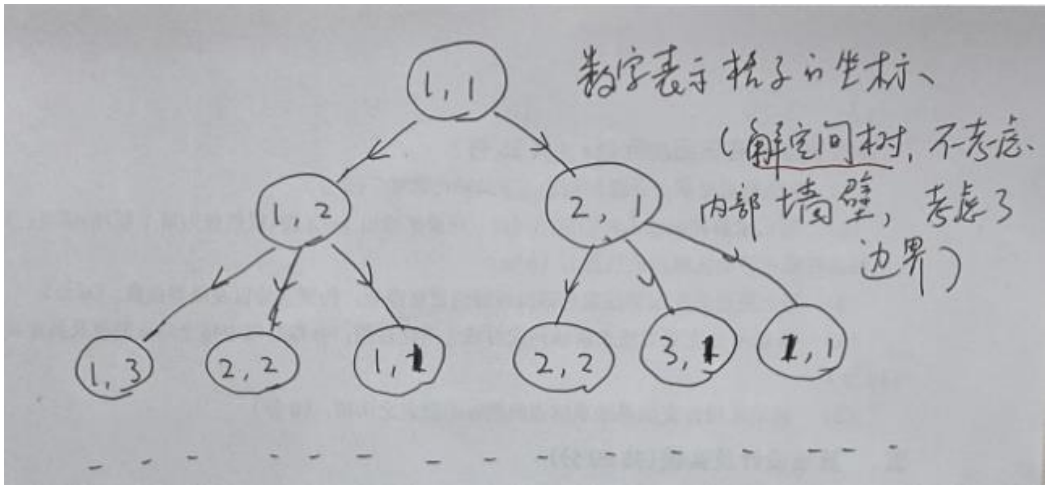
（设 i, j 是当前访问的格子的坐标，endI 和 endJ 是迷宫出口坐标）

（2）绘制该问题通用的解空间树（只画两层，根节点为第 0 层）(5 分)。

如果节点内不画节点坐标，则解空间树是标准四叉树：

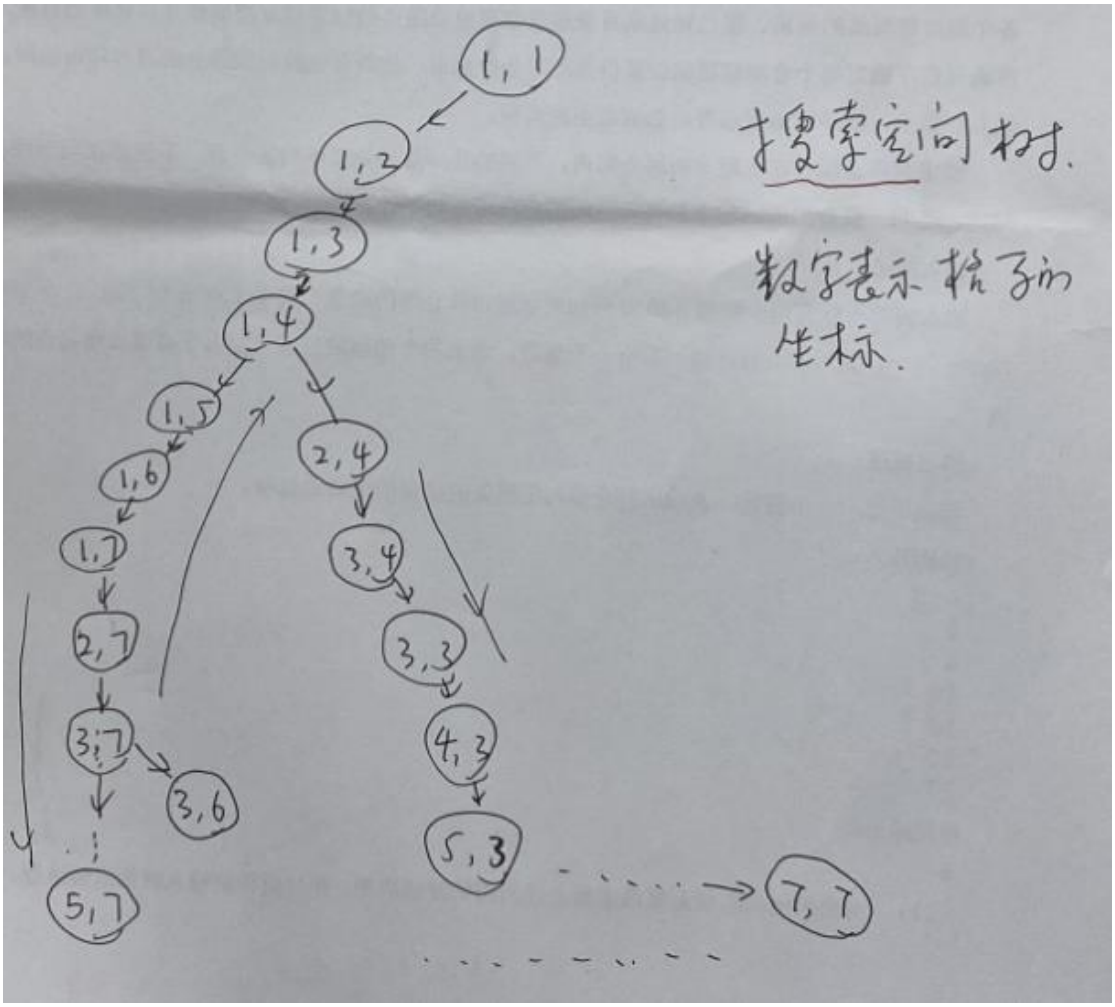


如果画了节点坐标（从入口（1，1）开始，每个格子从右侧开始按照顺时针搜索），可参考下图：



附：该问题实例的一条搜索空间树（从入口（1，1）开始，每个格子从“右侧开始按

照顺时针搜索”）：





（3）采用回溯法编写程序实现上述题目要求，附源代码，并给出以下测试用例的输出。（10 分）

参考代码：

```
#include <iostream>
#include <stdio.h>
int startI = 1, startJ = 1;
int endI = 7, endJ = 7;
int pathCount = 0, minLength = 65535, pathLength = 0;
```

```
int maze[9][9] = {
        { 2,2,2,2,2,2,2,2,2},
        { 2,0,0,0,0,0,0,0,2},
        { 2,0,2,0,2,2,2,0,2},
        { 2,0,2,0,0,2,0,0,2},
        { 2,0,2,0,2,0,2,0,2},
        { 2,0,0,0,0,0,2,0,2},
        { 2,2,0,2,2,0,2,2,2},
        { 2,0,0,0,0,0,0,0,2},
        { 2,2,2,2,2,2,2,2,2}
};
void visit(int i, int j)
{
        int m, n;
        maze[i][j] = 1;    //标记当前坐标已经被访问
        pathLength = pathLength + 1;
        if (i == endI && j == endJ)
        {
                pathCount = pathCount + 1;
                if (pathLength < minLength)
                        minLength = pathLength;
                printf("\n 路径(长度=%d): \n", pathLength);
                for (m = 0; m < 9; m++)
                {
                        for (n = 0; n < 9; n++)
                        {
                                if (maze[m][n] == 2)
                                {
                                        printf("*");
                                }
                                else if (maze[m][n] == 1)
                                        printf(".");
                                else
                                        printf(" ");
                        }
                        printf("\n");
                }
        }
        if (maze[i][j + 1] == 0)
        {
                visit(i, j + 1);
        }
        if (maze[i + 1][j] == 0) {
                visit(i + 1, j);
```

```
    }
    if (maze[i][j - 1] == 0)
    {
        visit(i, j - 1);
    }
    if (maze[i - 1][j] == 0)
    {
        visit(i - 1, j);
    }
    maze[i][j] = 0;     //恢复当前坐标未被访问，回溯
    pathLength = pathLength - 1;
}

int main(void)
{
    int i, j;
    visit(startI, startJ);
    printf("路径条数：%d\n", pathCount);
    printf("最小长度：%d\n", minLength);
    system("pause");
    return 0;
}
```

四、编程题 3：黑白格游戏（共 20 分）

某个人站在一个由黑白方格构成的矩形区域中，他从某个黑色方格出发，向上下左右四个方向移动，每次只能从一个黑色方格移动到另一个黑色方格，问他最多能够走多少个黑色方格。

输入：

输入数据的第一行包含两个整数 L 和 w，表示矩形区域的宽度（列数）和高度（行数），也就是水平和垂直方向上方格的数量。其后的 w 行，每一行有 L 个字符，由“.”,”#”和“@”符号构成，分别表示黑色方格，白色方格和人起始所在的黑色方格。

输出：

输出一个整数，表示他最多能走的黑色方格的数量，占一行。

要求：

（1）写出采用回溯法求解该问题时的约束条件和目标函数。（10 分）

约束函数：

（不能越过边界）并且 （只能访问黑色方格）并且 （每个黑色方格只访问一次）

注意：也可以用符号表示，如

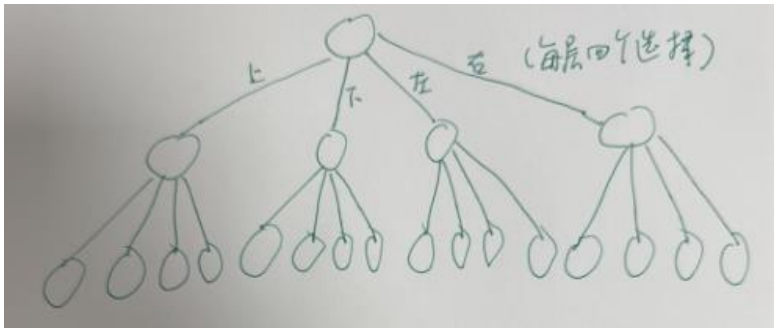
(i>=1) and (i<=n) and (j>=1) and (j<=n) and (map[i,j]==’.’) and !visited[i,j]

目标函数：

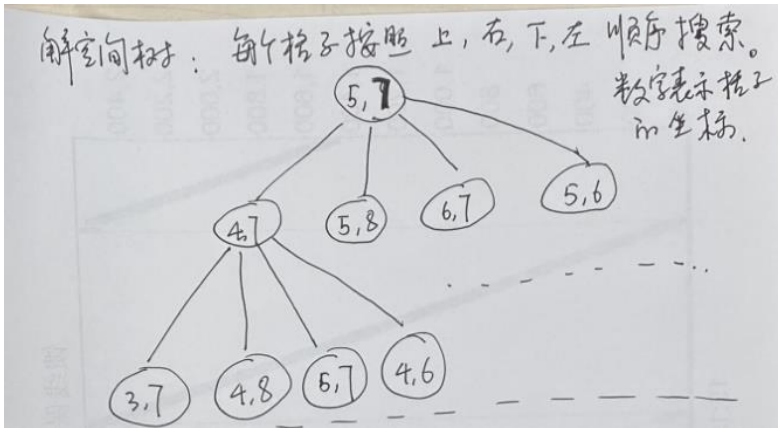
计算最多能走到的黑色方格数量

附一解空间树：

如果节点内不画节点坐标，则解空间树是标准四叉树：

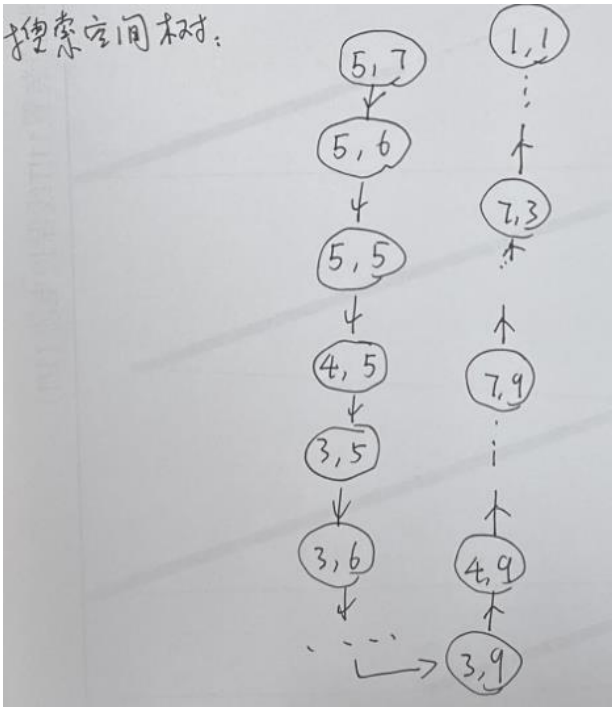


如果画了节点坐标，可参考下图：



附一左边样例的搜索空间树：

```
11 9
.#.........
.#.#######.
.#.#.....#.
.#.#.###.#.
.#.#..@#.#.
.#.#####.#.
.#.......#.
.#########.
...........
```



（2）采用回溯法编写程序实现上述题目要求，附源代码，并给出以下测试用例的输出。（10 分）

1、两个测试用例的输出分别为 45 和 59。

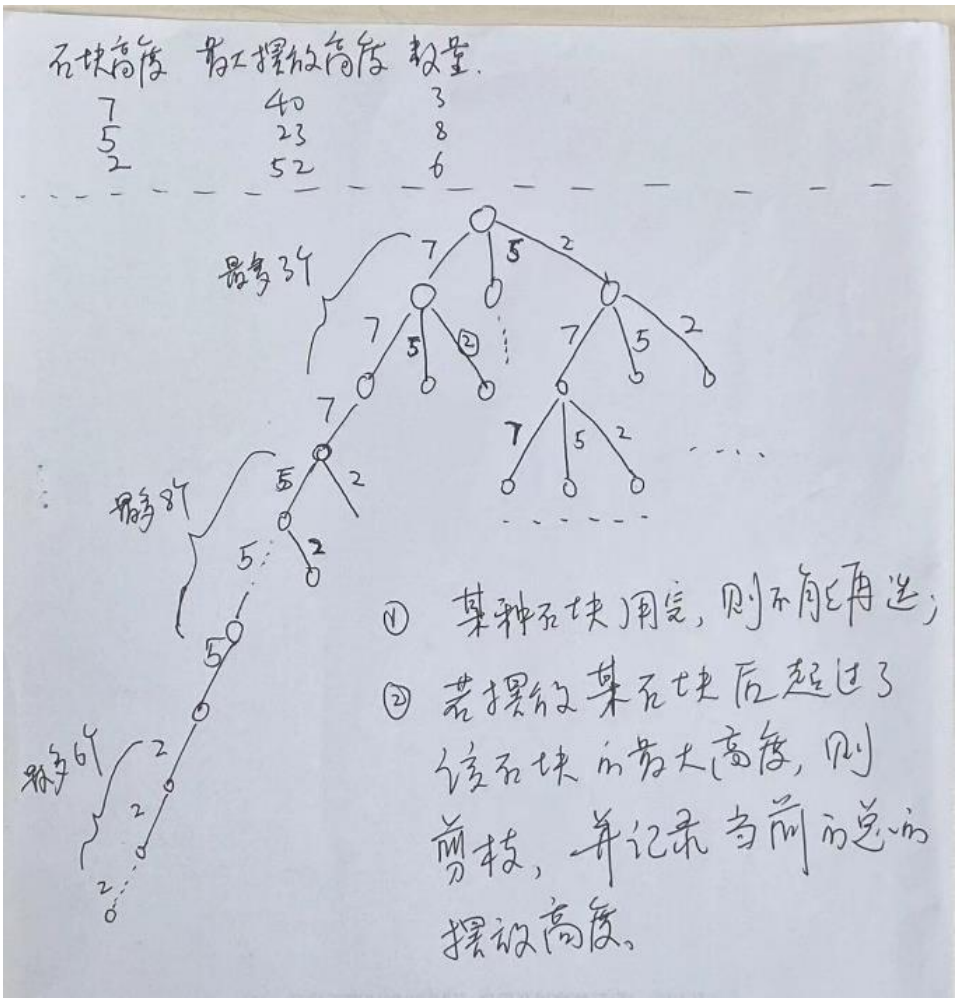
2、参考代码：

```
using namespace std;
int num;
char map[25][25];
int visited[25][25];
int dir[4][2] = {0, -1, 1, 0, 0, 1, -1, 0};   //方向向量，通过它可以直接得到下一个格子的坐标。
void DFS(int x, int y, int n, int m)
{
   int mx, my;
   for(int i = 0; i<4; i++)
   {
      mx = x+dir[i][0];
      my = y+dir[i][1];     //mx、my 是下一个格子的坐标
      if(mx>=1 && mx<=n && my>=1 && my<=m && !visited[mx][my] && map[mx][my]=='.')
      {
         visited[mx][my] = 1;
         num++;
         DFS(mx, my, n, m);
       //注意：该问题是计算能到达的格子的总数，而不是路径，因此每个格子全局只需被访问一次，访问后 num++，不能重复访问（即不需要类似于 num--，visited[][]=0 的回溯）。
      }
   }
}
int main()
{
   int n, m, x, y;
   while(scanf("%d%d", &m, &n)!=EOF && (n || m))
   {
      num = 1;
      memset(visited, 0, sizeof(visited));
      for(int i = 1; i<=n; i++)
      {
         for(int j = 1; j<=m; j++)
         {
            cin>>map[i][j];
            if(map[i][j] == '@')
            {
               x = i;
               y = j;
               visited[x][y] = 1;
```

```
            }
        }
    }
    DFS(x, y, n, m);
    printf("%d\n", num);
  }
}
```

五、编程题 4：修建高塔（选做，不占分）

解空间树（下图以样例输入为例绘制。如果只考虑有 n 类石块则可以画成标准的 n 叉树）：



注意：该问题的递归深度不是固定的。如果不再有满足条件的石块，则递归结束。

约束函数：（石块剩余数量>0） 并且 (石块的摆放高度<=该类石块的最大允许高度)
目标函数：Max（高塔的高度）

参考代码：

```
#include <iostream>
```

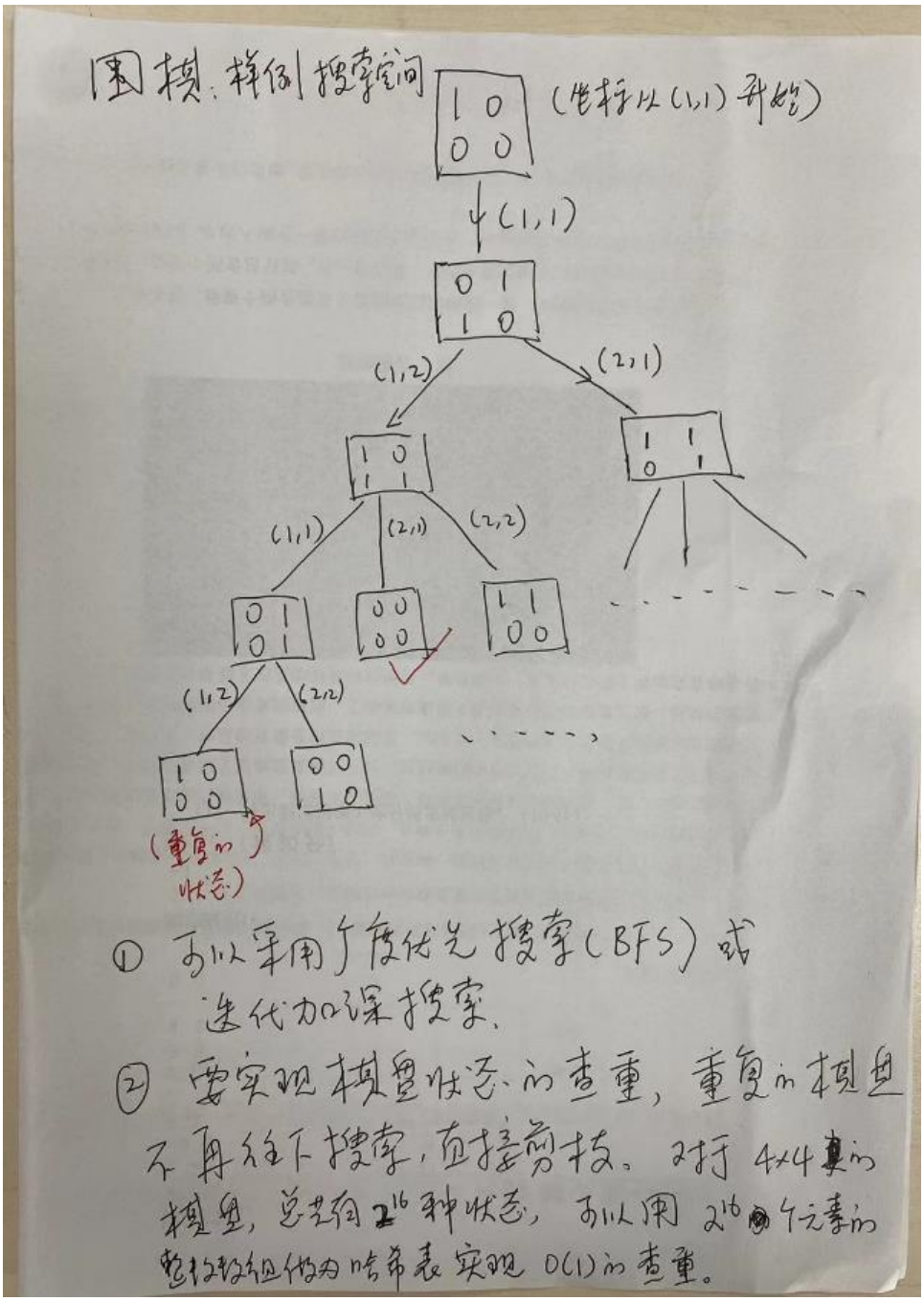
```
using namespace std;
int find_sol(int hi[],int ai[],int ci[],int yi[],int now_height,int n) {
    int my_max = -1;
    int temp;
    int flag0 = 0;
    for (int i = 0; i < n;i++) {
        //如果能找到满足条件的石块（石块有剩余，且该石块放置后高度满足要求）
        if (yi[i]>0 && now_height+hi[i]<=ai[i]) {
            yi[i]--;//石块数量减一
            temp=find_sol(hi, ai, ci, yi, now_height+ hi[i],n);//向下递归
            if (temp>my_max)//如果新的高度大于原来高度
            {
                my_max = temp;//更新
            }
            yi[i]++;//该石块数量加一
            flag0 = 1;//置标志为1,
        }
    }
    //如果标志为0，那么没有找到符合要求的，返回当前高度
    if (!flag0) {
        return now_height;
    }
    //否则找到石块，返回最大高度
    return my_max;
}
int main()
{
    int n;
    cin >> n;
    int* hi = new int[n];//第i种石块所占高度
    int* ai = new int[n];//第i种石块最大高度
    int* ci = new int[n];//第i种石块个数
    int* yi = new int[n];//ci保存剩余石块
    for (int i = 0; i < n;i++) {
        cin >> hi[i] >> ai[i] >> ci[i];
        yi[i] = ci[i];
    }
    cout << find_sol(hi, ai, ci, yi, 0, n) << endl;
    delete[]yi;
    delete[]ci;
    delete[]ai;
    delete[]hi;
    return 0;
}
```

六、编程题 5：黑白棋游戏（自学，不占分）

方法一：采用作业 2 中的代码（dfs 搜索），解空间树是标准的二叉树，深度是 m*n。
约束函数：（只能翻黑棋） 并且 （不用处理重复出现的棋盘） （本题是只能翻黑子）
目标函数：Min(使棋子达到全黑或全白的变换次数)

方法二：采用 BFS（广度优先搜索）。求解该问题 BFS 是更好的算法，但必须要解决棋盘状态存储和重复判断问题。可以采用状态压缩法存储棋盘状态。

一个 BFS 的例子如下：



BFS 参考代码：

```
#include <bits/stdc++.h>
#define INF 0x7fffffff
#define Maze vector<vector<bool> >
using namespace std;
int n, BM, ans = -1;
// 将棋盘称为图，当且仅当两个图中存在至少一个位置的黑白状态不同时，称两图不同
// 思路为对于每一种不同的图计算出从初始图到它需要的最少操作数
// 以下将一种图称为一种状态
int modify(int u, int pos) {
    // 对二进制数 u 表示的状态中 u 的第 pos 位（最低位为 0）对应的棋子进行操作
    u ^= 1 << pos; // 改变自身的颜色
    /*
        经过对棋盘转化为二进制数的操作，以 4*4 棋盘为例，以下给出棋盘中每个位置对应的二进制数中的下标
        15 14 13 12
        11 10  9  8
         7  6  5  4
         3  2  1  0
        可见，即二进制数中下标为 pos，pos%n 非零时，右侧存在相邻棋子，同理可得左侧、下侧、上侧存在相邻棋子的条件
    */
    if (pos % n != 0) u ^= 1 << pos - 1; // right
    if (pos % n != n - 1) u ^= 1 << pos + 1; // left
    if (pos >= n) u ^= 1 << pos - n; // down
    if (pos < BM - n) u ^= 1 << pos + n; // up;
    return u;
}
void bfs(Maze &mp) { // mp 代表起点，即初始状态
    int top_ = 1ll << BM, tmp = 0; // BM 为位置数量，即 n * n
    // tmp 为初始状态的二进制数表示
    for (int i = 1; i <= n; i++) for (int j = 1; j <= n; j++) tmp = tmp << 1 | mp[i][j];
    if (tmp == top_ - 1 || !tmp) {ans = 0; return;} // 无需操作
    // 为了将图去重，使用数据结构维护操作到每个状态的最小操作数
    // 有两种思路，一种是使用 map<Maze, int>，另一种是将图转化为二进制数后使用数组存储
    // 第一种操作更加直观，但会使复杂度乘上 O(log(2^(n*n)))，故采用第二种方案节省时间

    vector<int> f(top_, INF); f[tmp] = 0; // 此行中的 f 可视为 dp 数组，存储到达每个状态的最少操作数。到达初始图的操作数自然为 0。
    queue<int> que; que.push(tmp); // 初始状态入队
    for (int u, v; !que.empty(); que.pop()) {
        u = que.front();
```

```
            // 遍历当前状态的每一个黑色棋子，即二进制数表示中的每一个 1，穷举出该状态能转移到的所有可能的状态
            for (int i = 0, tmp = 1; i < BM; i++, tmp <<= 1) {
                if (u & tmp == 0) continue; // 白色棋子忽略
                v = modify(u, i); // 更改二进制数的第 i 位对应的棋子
                if (f[v] == INF) { // 第一次到达该状态
                    f[v] = f[u] + 1; // 更新答案
                    if (!v) {ans = f[v]; return;} // 二进制数 0 对应原图中每一格均为 0，即找到最终结果
                        // 在当前题目背景下，当且仅当初始棋盘全黑时才可能达到全黑，故此处无需考虑全黑
                    que.push(v); // 第一次到达该状态时，入队。
                }
                // 由于每个状态的最小操作数一定在初次被访问时就已经求得，故 f[v]不等于正无穷时无需考虑更新答案
            }
        }
}

int main() {
    string str;
    cin >> n; BM = n * n;
    Maze mp(n + 1, vector<bool>(n + 1));
    for (int i = 1; i <= n; i++) {
        cin >> str;
        for (int j = 1; j <= 4; j++) mp[i][j] = str[j - 1] == 'w' ? 0 : 1;
    }
    bfs(mp);
    cout << ans << '\n';
    return 0;
}
```

方法三、采用迭代加深搜索（本题采用 IDDFS 并不优化）

迭代加深搜索（Iterative Deepening Depth-First Search, IDDFS） 是一种结合了深度优先搜索（DFS）内存效率与广度优先搜索（BFS）最优性（在无权图中）的搜索算法。它通过逐渐增加深度限制来反复运行深度优先搜索，直到找到目标。

方法	优点	缺点
BFS	保证找到最短路径（无权图）	空间复杂度高 $O(b^d)$
DFS	空间复杂度低 $O(bd)$	可能陷入很深的分支，不保证最优
IDDFS	空间复杂度低 $O(bd)$，保证找到最短路径	节点会被重复扩展

参考代码：

```
#include <iostream>
#include <vector>
using namespace std;
const int maxn = 15;
vector<vector<char>> grid(maxn, vector<char>(maxn, 'b'));
int n;
void change(int x, int y){
    int dx[] = {0, 0, -1, 1, 0};
    int dy[] = {-1, 1, 0, 0, 0};
    for(int i = 0; i < 5; i++){
        int nx = x + dx[i];
        int ny = y + dy[i];
        if(grid[nx][ny] == 'b'){
            grid[nx][ny] = 'w';
        }
        else{
            grid[nx][ny] = 'b';
        }
    }

}
bool dfs(int x){    //深度受限的 DFS：迭代加深搜索，每次最多搜索 x 层
    if(x == 0){
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++){
                if(grid[i][j] != grid[1][1]) return false;
            }
        }
        return true;
    }

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            if(grid[i][j] == 'b'){
                change(i, j);   //注意：该代码未实现查重
                if(dfs(x - 1)) return true;
                change(i, j);   //再 change 一次完成撤销操作
            }
        }
    }
    return false;
}
```

```
int main(){
    cin >> n;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            cin >> grid[i][j];
        }
    }
    for(int i = 0; i <= 10; i++){
        if(dfs(i)){
            cout << i << endl;
            return 0;
        }
    }
    cout << "NO" << endl;
    return 0;
}
```